

SquareOps Interview — Debrief

Kubernetes · AWS · CI/CD · Session Manager | Sanket Chopade

■ The Self-Inflicted NLB Trap

This is the biggest problem. You mentioned NLB while answering an Ingress question — and the interviewer immediately exploited it with a 3-question follow-up chain. That's on you, not the interviewer.

When answering 'how does Ingress work,' NLB is an irrelevant tangent unless the question specifically asks about the full traffic path from outside AWS. Bringing it up unprompted signals you don't have clean mental separation between:

- **Ingress** → L7 routing inside the cluster (path/host-based → Service → Endpoints → Pods)
- **ALB/NLB** → AWS-level load balancers that sit in front of or alongside the cluster

The correct Ingress answer should have stayed inside the cluster boundary. The pod ephemerality follow-up is where you should have mentioned **kube-proxy + iptables/IPVS** and **Endpoints/EndpointSlices**. The Ingress controller watches Services → Services watch Endpoints → Endpoints are dynamically updated by kube-controller-manager as pods come and go.

■■ ALB vs NLB — Did You Actually Answer It Well?

Dimension	NLB (L4)	ALB (L7)
OSI Layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP/S)
Routing	IP + port only	Path, host, headers
SSL Termination	Pass-through	Native
AWS WAF	No	Yes
EKS Ingress	No L7 integration	AWS LBC provisions ALB
Use case	DB, gRPC, gaming	Microservices, web apps

Key point: AWS Load Balancer Controller provisions an ALB that natively integrates with Ingress resources — giving you path-based routing, SSL termination, and WAF. NLB can expose a NodePort/Service, but you lose all application-layer intelligence.

■■ Session Manager — You Handled It Correctly, But Still a Gap

Fix this now — it will come up again. Steps to enable SSM on EC2:

1. Attach IAM role with **AmazonSSMManagedInstanceCore** policy to the instance.
2. Ensure **SSM Agent is running** (pre-installed on Amazon Linux 2/2023, Ubuntu 16.04+).
3. Instance needs **outbound HTTPS** to SSM endpoints — either internet access OR VPC Interface Endpoints for ssm, ssmmessages, ec2messages.
4. No port 22. No PEM key. Works on private subnets via endpoints.

The VPC Endpoints detail separates a surface-level answer from a strong one — especially in private subnet scenarios, which is exactly the context where you'd lose PEM access.

■ Pod Failure End-to-End Flow — The Most Important Question

Expected answer — component by component:

- **kubelet**: Detects pod failure via liveness probe or container exit on the node.
- **API Server**: kubelet reports updated pod/node status.
- **ReplicaSet Controller**: Watches pod count (2 running vs 3 desired) → creates new Pod object.
- **Scheduler**: Assigns new pod to a node (resources, taints, affinity rules).
- **kubelet (target node)**: Pulls image, starts container.
- **CrashLoopBackOff**: kubelet applies exponential backoff: 10s → 20s → 40s → max 5 min.
- **Pending state**: Scheduler CANNOT place pod (no resources, PVC unbound, taint). Controller waits — nothing is evicted.

Critical distinction: CrashLoopBackOff = pod starts and crashes repeatedly. Pending = pod never starts. If you didn't distinguish these two states in your answer, that's a gap.

■ The 'Divergent Branch' Question at End-of-Interview

Asking the interviewer to explain their own question *after* the interview ends can read as: 'I didn't know the answer and I'm trying to retroactively understand what I missed.' Always ask for clarification **during** the question, not after.

A sharper closing question would have been:

```
"In your CI/CD setup, how do you handle long-lived feature branches that diverge significantly from main – do you enforce rebase policies or allow merge commits?"
```

■ Market Dominance Question — Reasonable Close

Expected answer:

- **EKS** dominates in enterprise AWS-heavy shops due to market share + IAM/VPC integration.
- **GKE** is technically superior — Google invented Kubernetes, leads upstream contributions.
- **AKS** wins in Microsoft/Azure ecosystem shops.
- EKS wins on **ecosystem lock-in**, not on technical merit.

Overall Signal Summary

Area	Signal	Action
NLB during Ingress answer	■ Self-inflicted trap	Keep answers scoped to the boundary asked
ALB vs NLB distinction	■■ Check your answer depth	Memorize the L4/L7 table above
Session Manager gap	■■ Handled honestly	Learn the 4 SSM enablement steps cold
Pod failure flow	■ Strongest signal	Ensure CrashLoopBackOff vs Pending distinction
Divergent branch question	■■ Timing risk	Ask for clarification during, not after

Market dominance close	■ Good initiative	Solid — shows you're thinking long-term
------------------------	-------------------	---

Key takeaway: You walked into at least one trap of your own making. The SSM gap was handled professionally. The pod failure flow and Ingress ephemerality questions are where the real signal was — those answers reveal whether you understand Kubernetes operationally or just theoretically.